

Побудова моделі

Мета

Ознайомитись з різновидами регресійних моделей. Після завершення цієї лабораторної роботи ви зможете:

- Будувати регресійні моделі кількох видів
- Оцінювати якість моделі візуально
- Оцінювати якість моделі за допомогою числових мір, без використання тестової вибірки
- Виконувати прогнозування відгуку, використовуючи побудовану модель

Завдання, що оцінюються

1. Скачати дані із файлу "clean_data2.csv" (Data2.csv з виправленими помилками та заповненими пропусками). Записати дані у dataframe. В попередній роботі ви визначили ознаки, що можуть бути предикторами для 'CO2 emission'. Побудуйте моделі лінійної регресії для кожного з цих предикторів.
2. Побудуйте модель множинної лінійної регресії для всіх доречних предикторів разом.
3. Побудуйте кілька поліноміальних моделей другого порядку.
4. Побудуйте візуалізації для оцінки всіх моделей.
5. Порахуйте значення R^2 та MSE для оцінки якості кожної моделі. Оберіть найкращу модель.

Завдання #1:

Скачати дані із файлу 'clean_data2.csv' (Data2.csv з виправленими помилками та заповненими пропусками). Записати дані у dataframe.

В попередній роботі ви визначили ознаки, що можуть бути предикторами для 'CO2 emission'. Побудуйте моделі лінійної регресії для кожного з цих предикторів.

Зчитую дані з файлу у датафрейм

```
In [2]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

DATA_PATH = 'data/clean_data.csv'

df = pd.read_csv(DATA_PATH, sep=';', encoding='cp1252')

df[['country_name', 'region']] = df[['country_name', 'region']].astype('
df.dtypes
```

```
Out[2]: country_name      string[python]
region                  string[python]
gdp_per_capita          int64
population              int64
co2_emission            float64
area                   float64
population_density      float64
dtype: object
```

З попередньої роботи знаємо, що хорошими показниками для 'CO2 emission' можуть бути:

- population
- area

Розробимо моделі, використовуючи ці ознаки як змінні-предиктори.
Створюю об'єкт лінійної регресії та навчаю першу модель

```
In [17]: from sklearn.linear_model import LinearRegression

lm = LinearRegression()
```

Знаходжу коефіцієнти моделі

```
In [18]: x = df[['population']]
y = df[['co2_emission']]
```

Будую лінійну модель та знаходжу прогноз

```
In [ ]: lm.fit(x, y)
y_hat1 = lm.predict(x)

y_hat1[0:5]
```

```
Out[ ]: array([[157961.46006296],
               [ 3792.80063443],
               [186825.7960797 ],
               [ -9784.64632044],
               [129618.38309565]])
```

Знаходжу коефіцієнти моделі

```
In [ ]: # a
lm.intercept_
```

```
Out[ ]: array([-10159.54669921])
```

```
In [21]: # b
lm.coef_
```

```
Out[21]: array([[0.00485113]])
```

Отримали кінцеву лінійну модель зі структурою:

$$\hat{Y}1 = a + bX$$

Підставляючи фактичні значення, маємо:

```
In [ ]: # Напишіть рівняння моделі, виконувати не потрібно
# co2_emission = -10159.54669921 + 0.00485113 * population
```

Створюю об'єкт лінійної регресії та навчаю другу модель

```
In [ ]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
lm2 = LinearRegression()
lm2

x = df[['area']]
y = df[['co2_emission']]

lm2.fit(x, y)

y_hat2 = lm2.predict(x)
y_hat2[0:5]
```

```
Out[ ]: array([[164572.96099962],
               [ 1535.03630769],
               [616212.90787019],
               [-5852.62373668],
               [319703.37262426]])
```

Знаходжу коефіцієнти моделі

```
In [23]: # a
lm2.intercept_
```

```
Out[23]: array([-5975.40309385])
```

```
In [24]: # b
lm2.coef_
```

```
Out[24]: array([[0.26123267]])
```

Отримали кінцеву лінійну модель зі структурою:

$$\hat{Y}2 = a + bX$$

Підставляючи фактичні значення, маємо:

```
In [ ]: # Напишіть рівняння моделі, виконувати не потрібно
# co2_emission = -5975.40309385 + 0.26123267 * area
```

Завдання #2:

Побудуйте модель множинної лінійної регресії для всіх доречних предикторів разом.

Створюю об'єкт лінійної регресії та навчаю множинну лінійну модель із структурою

$$\hat{Y} = a + b_1X_1 + b_2X_2$$

```
In [26]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
lm3 = LinearRegression()
lm3
```

```
Out[26]: ▼ LinearRegression ⓘ ⓘ
LinearRegression()
```

Знаходжу коефіцієнти моделі

```
In [27]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
z = df[['population', 'area']]
```

```
In [31]: # Будує модель
lm3.fit(z, y)

# a
print(lm3.intercept_)

# b
print(lm3.coef_)
```

```
[-64602.4231751]
[[0.00408248 0.12553168]]
```

Підставляючи фактичні значення, маємо:

```
In [ ]: # Напишіть рівняння моделі, виконувати не потрібно
# co2_emission = -64602.4231751 + 0.00408248 * population + 0.12553168 *
```

Завдання #3:

Побудуйте кілька поліноміальних моделей другого порядку.

Створюю об'єкт поліноміальної регресії та навчаю поліноміальну модель із структурою

$$\hat{Y}_4 = a + b_1 X_1 + b_2 X_1^2$$

Знаходжу коефіцієнти моделі

```
In [ ]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
import numpy as np

y = df['co2_emission']
x1 = df['population']

y_hat4 = np.polyfit(x1, y, 2)

pm = np.poly1d(y_hat4)
print(pm)
```

$$-4.064e-13 x^2 + 0.005361 x - 2.035e+04$$

```
In [40]: # Знаходжу коефіцієнти моделі

print(f'''Коефіцієнти моделі:
Коефіцієнт a: {y_hat4[2]}
Коефіцієнт b1: {y_hat4[1]}
Коефіцієнт b2: {y_hat4[0]}
''')
```

Коефіцієнти моделі:
Коефіцієнт a: -20353.248819624074
Коефіцієнт b1: 0.005361434280573773
Коефіцієнт b2: -4.0635020747943424e-13

$$\text{co2_emission} = -20353.248819624074 + 0.005361434280573773 * \text{population} +$$

$$-4.0635020747943424e-13 * \text{population}^2$$

Створюю об'єкт поліноміальної регресії та навчаю поліноміальну модель із структурою

$$\hat{Y}_5 = a + b_1 X_2 + b_2 X_2^2$$

Знаходжу коефіцієнти моделі

```
In [61]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання

x2 = df['area']
y_hat5 = np.polyfit(x2, y, 2)

pm2 = np.poly1d(y_hat5)
print(pm2)
```

$$-1.542e-08 x^2 + 0.4451 x - 6.59e+04$$

In [62]: `# Знаходжу коефіцієнти моделі`

```
print(f'''Коефіцієнти моделі:
Коефіцієнт a: {y_hat5[2]}
Коефіцієнт b1: {y_hat5[1]}
Коефіцієнт b2: {y_hat5[0]}
''')
```

Коефіцієнти моделі:
Коефіцієнт a: -65899.66785279519
Коефіцієнт b1: 0.4451438292736714
Коефіцієнт b2: -1.5417934961341924e-08

$$\text{co2_emission} = -65899.66785279519 + 0.4451438292736714 * \text{area} + \\ -1.5417934961341924e-08 * \text{area}^2$$

Створюю об'єкт поліноміальної регресії та навчаю поліноміальну модель із структурою

$$\hat{Y} = a + b_1 X_1 + b_2 X_2 + b_3 X_1 X_2 + b_4 X_1^2 + b_5 X_2^2$$

Знаходжу коефіцієнти моделі

In [43]: `# Напишіть ваш код нижче та натисніть Shift+Enter для виконання`
`from sklearn.preprocessing import PolynomialFeatures`

```
pf = PolynomialFeatures(degree=2)
pf
```

Out [43]: `PolynomialFeatures`

```
PolynomialFeatures()
```

In [44]: `z = df[['population', 'area']]`

```
z_pr = pf.fit_transform(z)
z.shape
```

Out [44]: (205, 2)

205 спостережень та 2 ознаки

In [45]: `z_pr.shape`

Out [45]: (205, 6)

Після трансформації отримали 205 спостережень та 6 ознак

In [46]: `y_hat6 = LinearRegression()`
`y_hat6.fit(z_pr, y)`

Out [46]:

```
▼ LinearRegression ⓘ ⓘ  
LinearRegression()
```

In [47]:

```
a = y_hat6.intercept_  
bs = y_hat6.coef_  
  
print(f"Коефіцієнт a: {a}")  
  
for b in range(1, len(bs)):  
    print(f"Коефіцієнт b{b}: {bs[b]}")
```

```
Коефіцієнт a: 1438.297851839161  
Коефіцієнт b1: 0.0023704702945945956  
Коефіцієнт b2: 0.02648874492661869  
Коефіцієнт b3: -3.177693145334876e-12  
Коефіцієнт b4: 1.0182694284710373e-09  
Коефіцієнт b5: -4.406939929454079e-09
```

```
co2_emissions = 1438.297851839161 + 0.0023704702945945956 * population +  
0.02648874492661869 * area + -3.177693145334876e-12 * population * area +  
1.0182694284710373e-09 * population^2 + -4.406939929454079e-09 * area^2
```

Завдання #4:

Побудуйте візуалізації для оцінки всіх моделей.

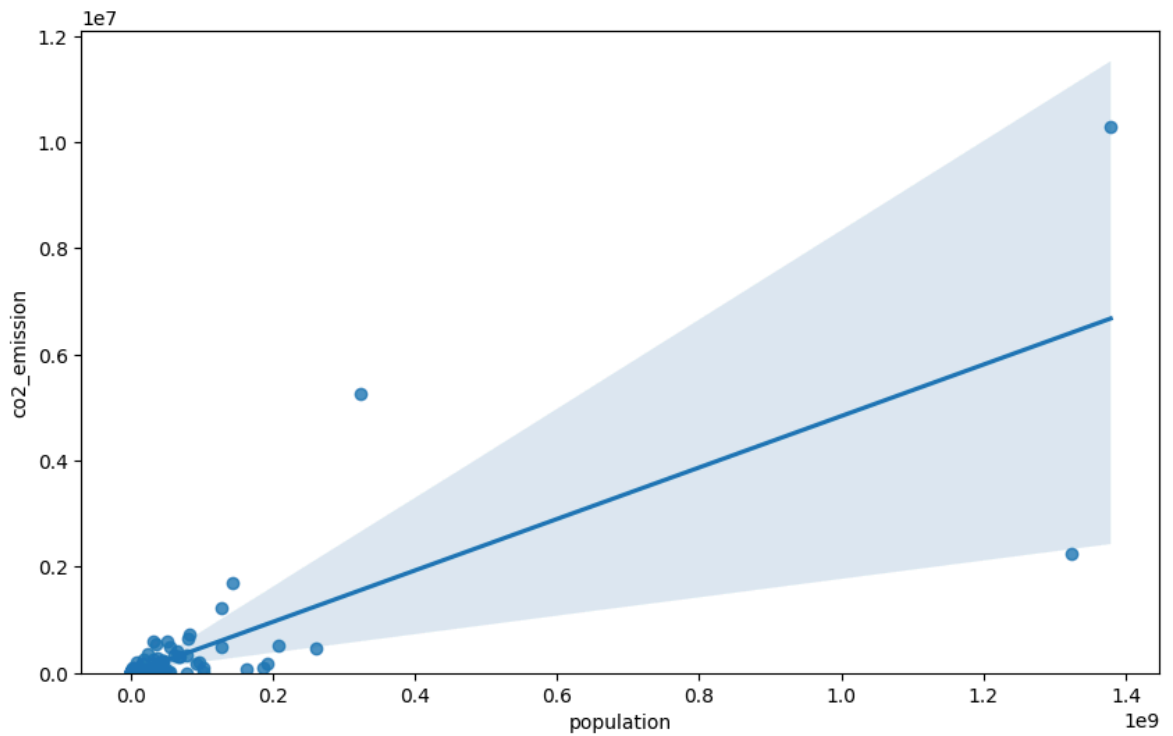
Для простої лінійної регресії чудовим способом візуалізації відповідності моделі є використання графіків регресії. Цей графік покаже комбінацію розсіяних точок даних (діаграма розсіювання, scatterplot) та підігнану лінію лінійної регресії, що проходить через дані.

Таким способом візуалізую перші моделі.

In [49]:

```
import seaborn as sns  
%matplotlib inline  
# Напишіть ваш код нижче та натисніть Shift+Enter для виконання  
  
plt.figure(figsize=(10, 6))  
sns.regplot(x='population', y='co2_emission', data=df)  
plt.ylim(0,)
```

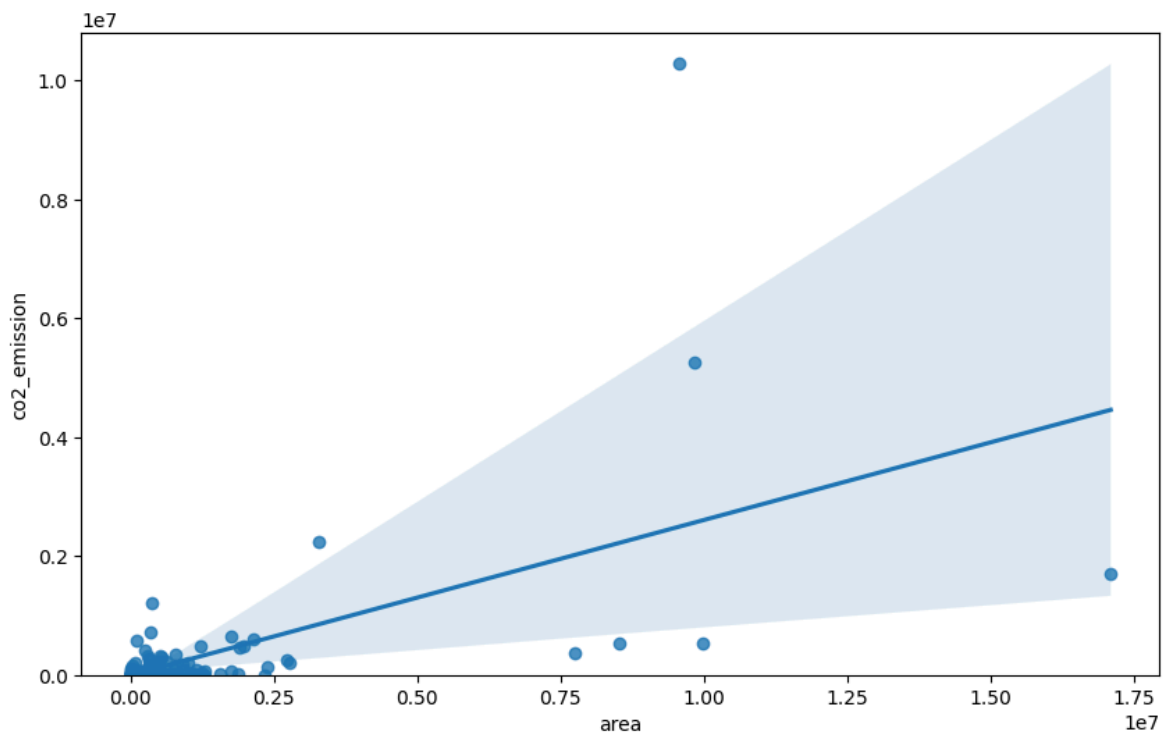
Out [49]: (0.0, 12106615.861098735)



Оскільки кут нахилу лінійної регресії позитивний, то зв'язок між чисельністю населення та обсягом викидів вуглекислого газу позитивний.

```
In [51]: plt.figure(figsize=(10, 6))
sns.regplot(x='area', y='co2_emission', data=df)
plt.ylim(0,)
```

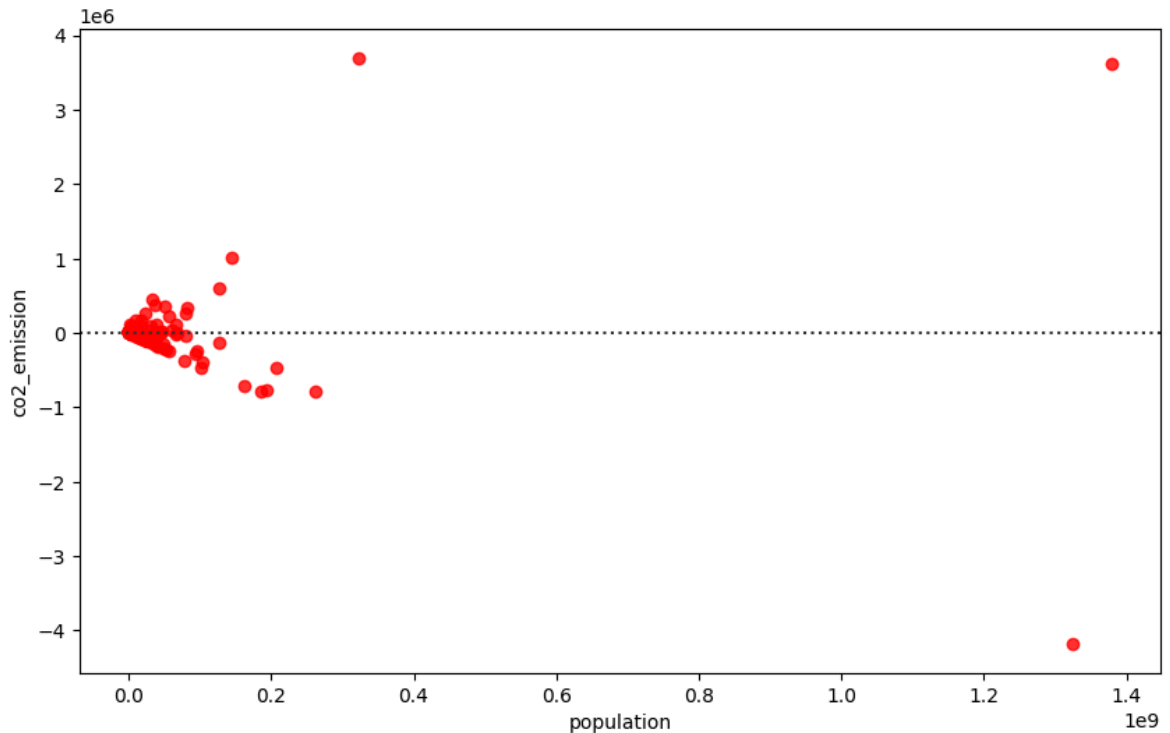
Out[51]: (0.0, 10813024.893512102)



Оскільки кут нахилу лінійної регресії позитивний, то зв'язок між площею та обсягом викидів вуглекислого газу позитивний.

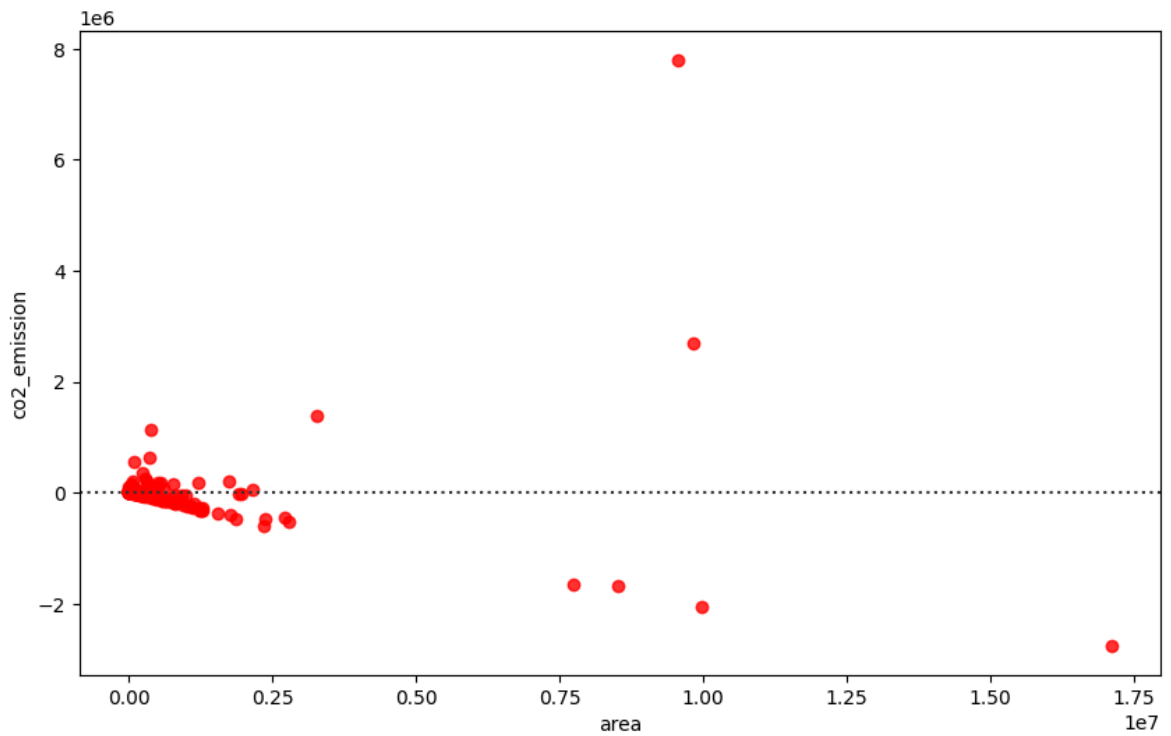
Для візуалізації дисперсії даних використаю діаграми залишків - по ній зможу прийняти рішення про необхідність ускладнення моделі.

```
In [53]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
plt.figure(figsize=(10, 6))
sns.residplot(x=df['population'], y=df['co2_emission'], color='red')
plt.show()
```



Оскільки на графіку залишків розподіл не є випадковий відносно осі абсцис, для аналізу краще використовувати нелінійну модель

```
In [54]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
plt.figure(figsize=(10, 6))
sns.residplot(x=df['area'], y=df['co2_emission'], color='red')
plt.show()
```



Оскільки на графіку залишків розподіл не є випадковий відносно осі абсцис, для аналізу краще використовувати нелінійну модель

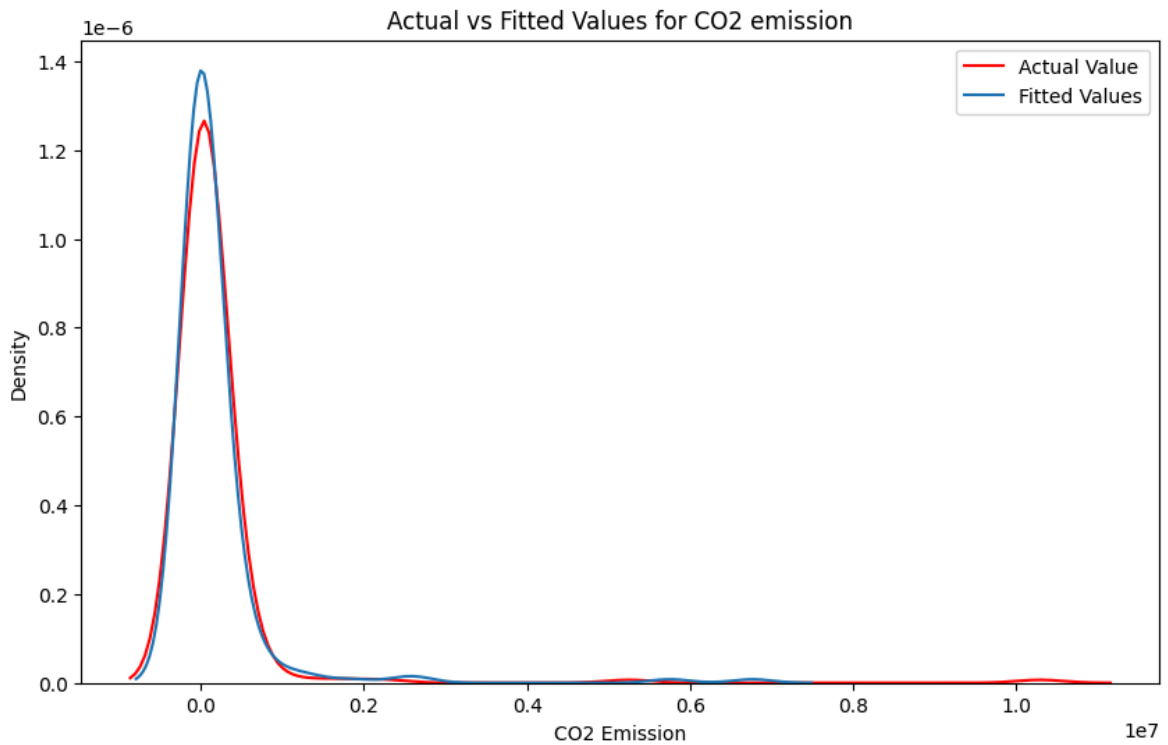
Для візуалізації моделі множинної лінійної регресії використаю діаграму розподілу.

```
In [ ]: plt.figure(figsize=(10, 6))
ax1 = plt.gca()

sns.kdeplot(df['co2_emission'], color="r", label="Actual Value", ax=ax1)

y_hat = lm3.predict(df[['population', 'area']])
sns.kdeplot(y_hat, color="b", label="Fitted Values", ax=ax1)

# Додаткове оформлення
plt.title("Actual vs Fitted Values for CO2 emission")
plt.xlabel("CO2 Emission")
plt.ylabel("Density")
plt.legend()
plt.show()
plt.close()
```



Підібрані значення та фактичні майже накладаються, тому вони досить близькі.

Для візуалізації поліноміальних моделей з одним предиктором використаю функцію `poly1d`. Зручно розміщувати всі моделі на одному графіку для обрання найкращої.

```
In [64]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
print(np.poly1d(y_hat4))
```

$$-4.064e-13 x^2 + 0.005361 x - 2.035e+04$$

```
In [63]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
print(np.poly1d(y_hat5))
```

$$-1.542e-08 x^2 + 0.4451 x - 6.59e+04$$

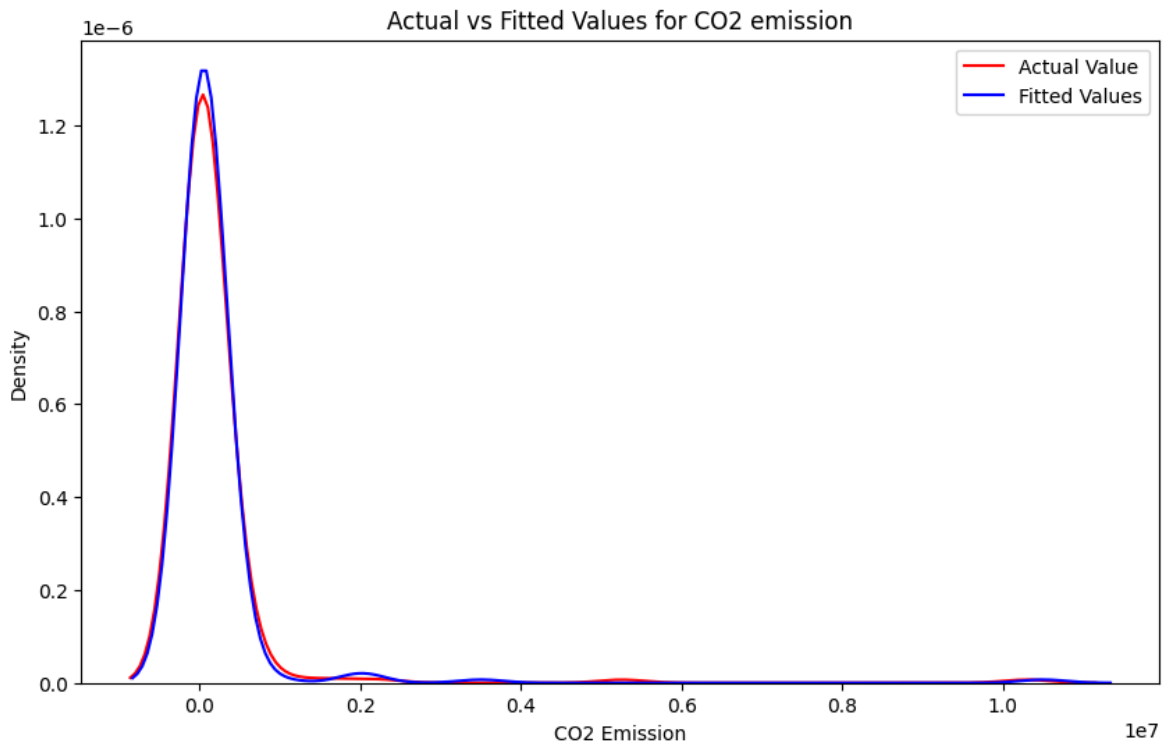
Для візуалізації поліноміальних моделей з кількома предикторами використаю діаграму розподілу або діаграму залишків

```
In [65]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
plt.figure(figsize=(10, 6))
ax1 = plt.gca()

sns.kdeplot(df['co2_emission'], color="r", label="Actual Value", ax=ax1)
y_hat = y_hat6.predict(z_pr)
sns.kdeplot(y_hat, color="b", label="Fitted Values", ax=ax1)

# Додаткове оформлення
plt.title("Actual vs Fitted Values for CO2 emission")
plt.xlabel("CO2 Emission")
plt.ylabel("Density")
plt.legend()
```

```
plt.show()
plt.close()
```



Завдання #5:

Порахуйте значення R^2 та MSE для оцінки якості кожної моделі. Оберіть найкращу модель.

Значення R^2 отримують безпосередньо з моделі `.score(X, Y)`, а для розрахунку MSE спочатку формують прогнозовані значення `.predict(X)` і порівнюють їх з фактичними

Модель 1:

$$\hat{Y}_1 = a + bX$$

```
In [ ]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
x1 = df[['population']]
y = df[['co2_emission']]

print('The R-square is: ', lm.score(x1, y))
```

The R-square is: 0.6461006246187183

65% варіації рівня викидів вуглекислого газу пояснюється $\hat{Y}_1$

```
In [68]: yhat1 = lm.predict(x1)
print('Output: ', yhat1[0:5])
```

```
Output: [[157961.46006296]
 [ 3792.80063443]
 [186825.7960797 ]
 [ -9784.64632044]
 [129618.38309565]]
```

```
In [69]: from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Calculate the mean squared error
mse = mean_squared_error(y, yhat1)
print('The mean squared error is: ', mse)

# Calculate the mean absolute error
mae = mean_absolute_error(y, yhat1)
print('The mean absolute error is: ', mae)

# Calculate the R-squared value
r2 = r2_score(y, yhat1)
print('The R-squared value is: ', r2)
```

```
The mean squared error is: 244683403840.77405
The mean absolute error is: 132907.01112251525
The R-squared value is: 0.6461006246187183
```

Модель 2:

$$\hat{Y}2 = a + bX$$

```
In [76]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
x2 = df[['area']]

print('The R-square is: ', lm2.score(x2, y))
```

```
The R-square is: 0.3451331940921192
```

35% варіації рівня викидів вуглекислого газу пояснюється $\hat{Y}2$

```
In [77]: yhat2 = lm2.predict(x2)
print('Output: ', yhat2[0:5])
```

```
Output: [[164572.96099962]
 [ 1535.03630769]
 [616212.90787019]
 [ -5852.62373668]
 [319703.37262426]]
```

```
In [84]: # Calculate the mean squared error
mse = mean_squared_error(y, yhat2)
print('The mean squared error is: ', mse)

# Calculate the mean absolute error
mae = mean_absolute_error(y, yhat2)
print('The mean absolute error is: ', mae)

# Calculate the R-squared value
r2 = r2_score(y, yhat2)
print('The R-squared value is: ', r2)
```

The mean squared error is: 452770053519.4301
The mean absolute error is: 178668.3942274042
The R-squared value is: 0.3451331940921192

Модель 3:

$$\hat{Y}_3 = a + b_1 X_1 + b_2 X_2$$

```
In [79]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
z = df[['population', 'area']]

print('The R-square is: ', lm3.score(z, y))
```

The R-square is: 0.7095760957246346

71% варіації рівня викидів вуглекислого газу пояснюється $\hat{Y}_3$

```
In [80]: y_predict_multiple = lm3.predict(z)
print('Output: ', y_predict_multiple[0:5])
```

Output: [[158834.74031486]
[-49251.76306514]
[400154.78456147]
[-64227.92516148]
[209528.30180787]]

```
In [81]: # Calculate the mean squared error
mse = mean_squared_error(y, y_predict_multiple)
print('The mean squared error is: ', mse)
# Calculate the mean absolute error
mae = mean_absolute_error(y, y_predict_multiple)
print('The mean absolute error is: ', mae)
# Calculate the R-squared value
r2 = r2_score(y, y_predict_multiple)
print('The R-squared value is: ', r2)
```

The mean squared error is: 200796933812.79166
The mean absolute error is: 161276.78453860086
The R-squared value is: 0.7095760957246346

Модель 4:

$$\hat{Y}_4 = a + b_1 X_1 + b_2 X_1^2$$

```
In [83]: p = np.poly1d(y_hat4)

r_squared = r2_score(y, p(x1))

print('The R-square is: ', r_squared)
```

The R-square is: 0.646669597790839

65% варіації рівня викидів вуглекислого газу пояснюється $\hat{Y}_4$

```
In [85]: # Calculate the mean squared error
mse = mean_squared_error(y, p(x1))
print('The mean squared error is: ', mse)
# Calculate the mean absolute error
mae = mean_absolute_error(y, p(x1))
print('The mean absolute error is: ', mae)
```

```
# Calculate the R-squared value
r2 = r2_score(y, p(x1))
print('The R-squared value is: ', r2)
```

The mean squared error is: 244290020008.7213
The mean absolute error is: 138798.7570131029
The R-squared value is: 0.646669597790839

Модель 5:

$$\hat{Y}_5 = a + b_1 X_2 + b_2 X_2^2$$

```
In [86]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
p = np.poly1d(y_hat5)
r_squared = r2_score(y, p(x2))
print('The R-square is: ', r_squared)
```

The R-square is: 0.3740137141949029

37% варіації рівня викидів вуглекислого газу пояснюється $\hat{Y}_5$

```
In [87]: # Calculate the mean squared error
mse = mean_squared_error(y, p(x2))
print('The mean squared error is: ', mse)
# Calculate the mean absolute error
mae = mean_absolute_error(y, p(x2))
print('The mean absolute error is: ', mae)
# Calculate the R-squared value
r2 = r2_score(y, p(x2))
print('The R-squared value is: ', r2)
```

The mean squared error is: 432802276080.3553
The mean absolute error is: 220627.8246147237
The R-squared value is: 0.3740137141949029

Модель 6:

$$\hat{Y}_6 = a + b_1 X_1 + b_2 X_2 + b_3 X_1 X_2 + b_4 X_1^2 + b_5 X_2^2$$

```
In [89]: # Напишіть ваш код нижче та натисніть Shift+Enter для виконання
r_squared = y_hat6.score(z_pr, y)
print('The R-square is: ', r_squared)
```

The R-square is: 0.938000210835021

94% варіації рівня викидів вуглекислого газу пояснюється $\hat{Y}_6$

```
In [90]: # Calculate the mean squared error
mse = mean_squared_error(y, y_hat6.predict(z_pr))
print('The mean squared error is: ', mse)
# Calculate the mean absolute error
mae = mean_absolute_error(y, y_hat6.predict(z_pr))
print('The mean absolute error is: ', mae)
# Calculate the R-squared value
r2 = r2_score(y, y_hat6.predict(z_pr))
print('The R-squared value is: ', r2)
```

The mean squared error is: 42866194476.758514

The mean absolute error is: 78155.41380847397

The R-squared value is: 0.938000210835021

Висновок

Порівнюючи всі моделі, роблю висновок, що **модель y_{hat6} є найкращою моделлю** для прогнозування 'CO2 emission' на основі нашого набору даних.

Додаткове завдання:

Побудуйте та оцініть кілька поліноміальних моделей різних порядків.

1. Побудуйте кілька поліноміальних моделей різних порядків.
2. Побудуйте візуалізації для оцінки всіх моделей (зручно розміщувати всі моделі на одному графіку для обрання найкращої).
3. Порахуйте значення R^2 та MSE для оцінки якості кожної моделі (теж доцільно побудувати графік залежності R^2 або MSE від порядку поліному моделі).
4. Оберіть найкращу модель, висновок обґрунтуйте.

```
In [91]: from sklearn.metrics import mean_squared_error, r2_score
```

```
# Для зручності обчислення
x = df['population']
y = df['co2_emission']

# Порядки поліномів, які будемо тестувати
degrees = [1, 2, 3, 4, 5]

# Словники для збереження метрик
models = {}
mse_values = []
r2_values = []

# Побудова моделей
for d in degrees:
    coeffs = np.polyfit(x, y, d)
    model = np.poly1d(coeffs)
    y_pred = model(x)

    mse = mean_squared_error(y, y_pred)
    r2 = r2_score(y, y_pred)

    models[d] = model
    mse_values.append(mse)
    r2_values.append(r2)

print(f"Порядок: {d}")
print(f" $R^2$  = {r2:.4f}, MSE = {mse:.2f}")
print(model, "\n")
```


Порядок: 1
 $R^2 = 0.6461$, $MSE = 244683403840.77$

$$0.004851 x - 1.016e+04$$

Порядок: 2
 $R^2 = 0.6467$, $MSE = 244290020008.72$
$$-4.064e-13 x^2 + 0.005361 x - 2.035e+04$$

Порядок: 3
 $R^2 = 0.6731$, $MSE = 225999777902.84$
$$2.085e-20 x^3 - 3.278e-11 x^2 + 0.01103 x - 8.091e+04$$

Порядок: 4
 $R^2 = 0.8711$, $MSE = 89148889556.12$
$$9.925e-29 x^4 - 1.991e-19 x^3 + 9.474e-11 x^2 - 0.004974 x + 6.646e+04$$

Порядок: 5
 $R^2 = 0.9359$, $MSE = 44299321776.31$
$$4.06e-37 x^5 - 1.142e-27 x^4 + 9.445e-19 x^3 - 2.009e-10 x^2 + 0.01418 x - 5.072e+04$$

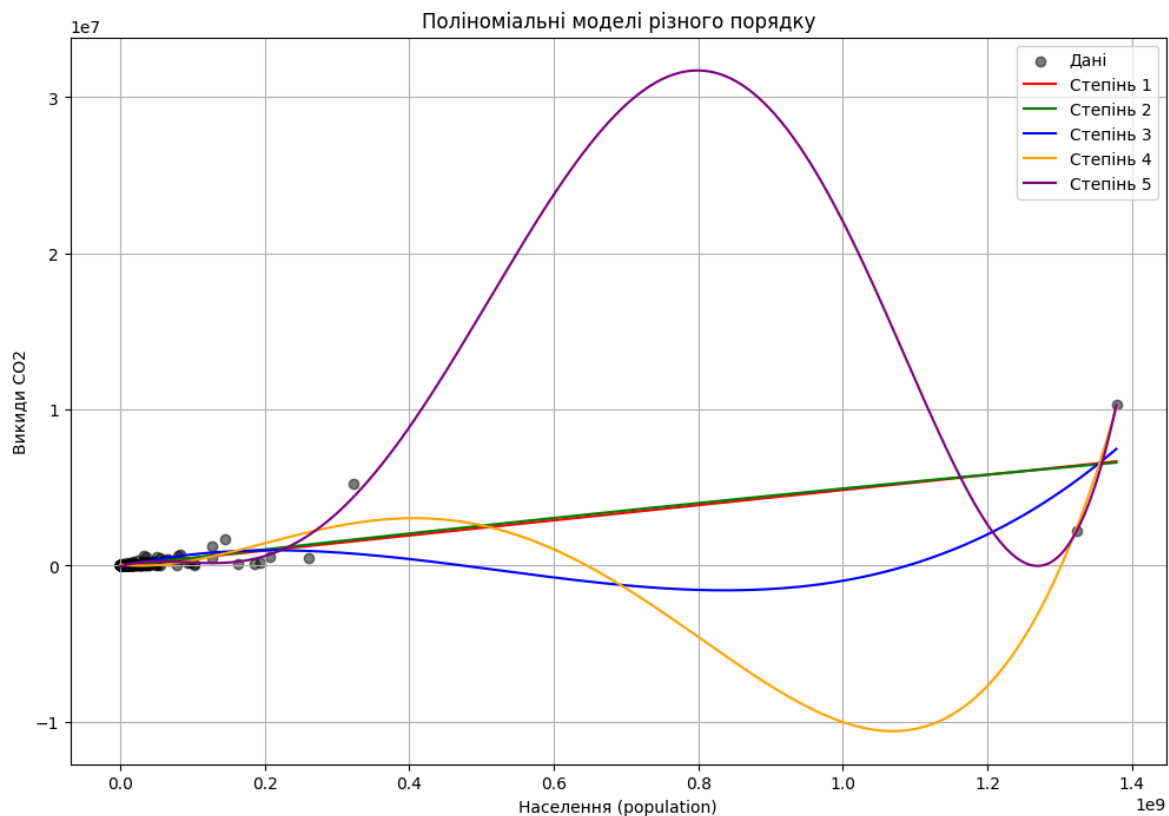
```
In [92]: plt.figure(figsize=(12, 8))
plt.scatter(x, y, color='black', label='Дані', alpha=0.5)

x_vals = np.linspace(x.min(), x.max(), 500)

# Кольори для графіків
colors = ['r', 'g', 'b', 'orange', 'purple']

for i, d in enumerate(degrees):
    y_vals = models[d](x_vals)
    plt.plot(x_vals, y_vals, label=f'Ступінь {d}', color=colors[i])

plt.xlabel("Населення (population)")
plt.ylabel("Викиди CO2")
plt.title("Поліноміальні моделі різного порядку")
plt.legend()
plt.grid(True)
plt.show()
```

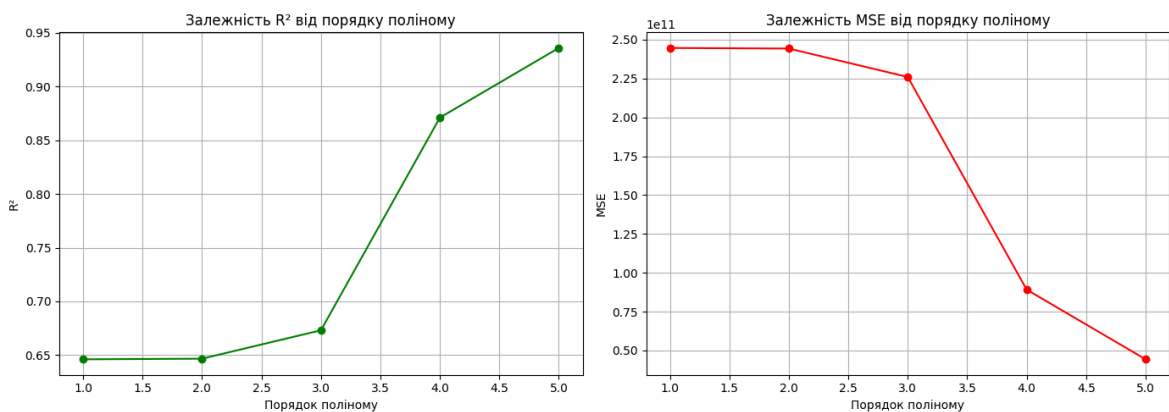


```
In [93]: plt.figure(figsize=(14, 5))

# R² графік
plt.subplot(1, 2, 1)
plt.plot(degrees, r2_values, marker='o', linestyle='--', color='green')
plt.title('Залежність R² від порядку поліному')
plt.xlabel('Порядок поліному')
plt.ylabel('R²')
plt.grid(True)

# MSE графік
plt.subplot(1, 2, 2)
plt.plot(degrees, mse_values, marker='o', linestyle='--', color='red')
plt.title('Залежність MSE від порядку поліному')
plt.xlabel('Порядок поліному')
plt.ylabel('MSE')
plt.grid(True)

plt.tight_layout()
plt.show()
```



```
In [94]: best_degree = degrees[np.argmax(r2_values)]  
         print(f"Найкраща модель – поліном порядку {best_degree}")
```

Найкраща модель – поліном порядку 5

Виконав студент групи ІП-33
Соколов Олександр